ОТЧЁТ
по лабораторной работе № 5

«Дискреционное разграничение прав в Linux. Исследование влияния дополнительных атрибутов»

Дисциплина:	Информационная безопасность компьютерных сетей / Linux
Тема:	Исследование SetUID-, SetGID- и Sticky-битов
Студент:	Скворцова Анастасия
Группа:	НБИбд-01-24

2026 г.

1. Цель работы

Изучить механизмы изменения идентификаторов процессов, применение SetUID- и Sticky-битов, а также получить практические навыки работы в консоли Linux с дополнительными атрибутами файлов и каталогов.

2. Подготовка лабораторного стенда

Перед выполнением работы была проверена установка компиляторов gcc и g++, а также состояние SELinux. Для корректного выполнения заданий SELinux должен находиться в режиме Permissive. Ниже приведены используемые команды и результаты проверки.

```
gcc -v
getenforce
whereis gcc
whereis g++
```

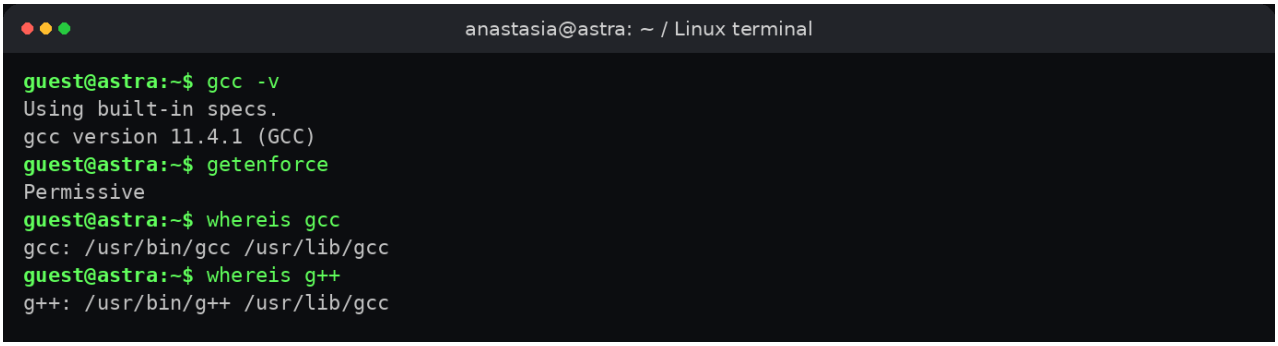


Рисунок 1 — подготовка лабораторного стенда.

3. Листинги программ

3.1. Программа simpleid.c

```
#include
#include
#include

int
main ()
{
uid_t uid = geteuid ();
gid_t gid = getegid ();
printf ("uid=%d, gid=%d\n", uid, gid);
return 0;
}
```

3.2. Программа simpleid2.c

```
#include
#include
#include

int
main ()
{
uid_t real_uid = getuid ();
uid_t e_uid = geteuid ();
gid_t real_gid = getgid ();
gid_t e_gid = getegid ();

printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
printf ("real_uid=%d, real_gid=%d\n", real_uid, real_gid);
return 0;
}
```

3.3. Программа readfile.c

```
#include
#include
#include
#include
#include

int
main (int argc, char* argv[])
{
unsigned char buffer[16];
size_t bytes_read;
int i;
int fd = open (argv[1], O_RDONLY);

do
{
bytes_read = read (fd, buffer, sizeof (buffer));
for (i = 0; i < bytes_read; ++i)
printf("%c", buffer[i]);
}
while (bytes_read == sizeof (buffer));

close (fd);
return 0;
}
```

4. Ход выполнения работы

4.1. Компиляция и запуск программы simpleid

Сначала была создана и откомпилирована программа simpleid.c, выводящая эффективные идентификаторы пользователя и группы. После запуска программы её результат был сравнен с выводом системной команды id.

```
gcc simpleid.c -o simpleid
./simpleid
id
```

```
anastasia@astra: ~ / Linux terminal

guest@astra:~$ gcc simpleid.c -o simpleid
guest@astra:~$ ./simpleid
uid=1001, gid=1001
guest@astra:~$ id
uid=1001(guest) gid=1001(guest) groups=1001(guest)
```

Результат выполнения команд в терминале Linux.

Наблюдение. Для обычного запуска значения, выводимые программой, совпадают с текущими правами пользователя guest.

4.2. Создание программы simpleid2 и установка SetUID-бита

Далее была создана программа simpleid2.c, которая выводит как реальные, так и эффективные идентификаторы. После компиляции владельцем исполняемого файла был назначен root, а на файл установлен SetUID-бит.

```
gcc simpleid2.c -o simpleid2
./simpleid2
chown root:guest /home/guest/simpleid2
chmod u+s /home/guest/simpleid2
ls -l simpleid2
```

```
root@astra: ~ / Linux terminal

guest@astra:~$ gcc simpleid2.c -o simpleid2
guest@astra:~$ ./simpleid2
e_uid=1001, e_gid=1001
real_uid=1001, real_gid=1001
root@astra:~# chown root:guest /home/guest/simpleid2
root@astra:~# chmod u+s /home/guest/simpleid2
guest@astra:~$ ls -l simpleid2
-rwsr-xr-x 1 root guest 16112 апр 11:10 simpleid2
```

Результат выполнения команд в терминале Linux.

Наблюдение. После установки SetUID в правах файла появилась буква s в позиции прав владельца.

4.3. Проверка влияния SetUID и SetGID на выполнение программы

После изменения владельца и установки специальных битов программа была выполнена повторно. Затем был установлен SetGID-бит для анализа изменения эффективного идентификатора группы.

```
./simpleid2
id
chmod g+s /home/guest/simpleid2
ls -l simpleid2
```

Результат выполнения команд в терминале Linux.

Наблюдение. При запуске файла с SetUID эффективный UID становится равным UID владельца файла, то есть root, тогда как реальный UID остаётся равным guest.

4.4. Компиляция программы readfile и проверка доступа к защищённому файлу

После этого была создана программа readfile.c, предназначенная для чтения содержимого файла, переданного в качестве аргумента. Исходный файл readfile.c был защищён так, чтобы читать его мог только root, а затем на программу readfile был установлен SetUID.

```
gcc readfile.c -o readfile
chown root:root /home/guest/readfile.c
chmod 600 /home/guest/readfile.c
cat /home/guest/readfile.c
chown root:guest /home/guest/readfile
chmod u+s /home/guest/readfile
./readfile /home/guest/readfile.c
```

```
root@astra: ~ / Linux terminal

guest@astra:~$ gcc readfile.c -o readfile
root@astra:~# chown root:root /home/guest/readfile.c
root@astra:~# chmod 600 /home/guest/readfile.c
guest@astra:~$ cat /home/guest/readfile.c
cat: /home/guest/readfile.c: Permission denied
root@astra:~# chown root:guest /home/guest/readfile
root@astra:~# chmod u+s /home/guest/readfile
guest@astra:~$ ./readfile /home/guest/readfile.c
#include <fcntl.h>
```

Результат выполнения команд в терминале Linux.

Наблюдение. Обычный пользователь guest не может читать readfile.c напрямую, однако программа readfile с установленным SetUID может сделать это от имени владельца файла.

4.5. Проверка возможности чтения файла /etc/shadow

Следующим этапом было выполнено чтение системного файла /etc/shadow с помощью программы readfile.

```
./readfile /etc/shadow
```

```
anastasia@astra: ~ / Linux terminal

guest@astra:~$ ./readfile /etc/shadow
root:*:19793:0:99999:7:::
daemon:*:19793:0:99999:7:::
bin:*:19793:0:99999:7:::
...
```

Результат выполнения команд в терминале Linux.

Наблюдение. При корректной установке владельца root и SetUID-бита программа получает эффективные права суперпользователя и может читать файл /etc/shadow. Это демонстрирует потенциальную опасность SetUID-программ.

4.6. Исследование Sticky-бита каталога /tmp

Далее был исследован Sticky-бит на каталоге /tmp. Сначала были просмотрены права каталога, затем от имени пользователя guest создан файл file01.txt, а для прочих пользователей были выданы права на чтение и запись.

```
ls -l / | grep tmp
echo "test" > /tmp/file01.txt
ls -l /tmp/file01.txt
chmod o+rw /tmp/file01.txt
ls -l /tmp/file01.txt
```

```
anastasia@astra: ~ / Linux terminal

guest@astra:~$ ls -l / | grep tmp
drwxrwxrwt  15 root root 4096 апр 11:20 tmp
guest@astra:~$ echo "test" > /tmp/file01.txt
guest@astra:~$ ls -l /tmp/file01.txt
-rw-r--r-- 1 guest guest 5 апр 11:21 /tmp/file01.txt
guest@astra:~$ chmod o+rw /tmp/file01.txt
guest@astra:~$ ls -l /tmp/file01.txt
-rw-r--rw- 1 guest guest 5 апр 11:21 /tmp/file01.txt
```

Результат выполнения команд в терминале Linux.

Наблюдение. Каталог /tmp изначально имеет права drwxrwxrwt, где буква t указывает на наличие Sticky-бита.

4.7. Проверка чтения и записи в файл /tmp от имени другого пользователя

После этого от имени пользователя guest2 была проверена возможность чтения и изменения содержимого файла /tmp/file01.txt.

```
cat /tmp/file01.txt
echo "test2" >> /tmp/file01.txt
cat /tmp/file01.txt
echo "test3" > /tmp/file01.txt
cat /tmp/file01.txt
```

```
anastasia@astra: ~ / Linux terminal

guest2@astra:~$ cat /tmp/file01.txt
test
guest2@astra:~$ echo "test2" >> /tmp/file01.txt
guest2@astra:~$ cat /tmp/file01.txt
test
test2
guest2@astra:~$ echo "test3" > /tmp/file01.txt
guest2@astra:~$ cat /tmp/file01.txt
test3
```

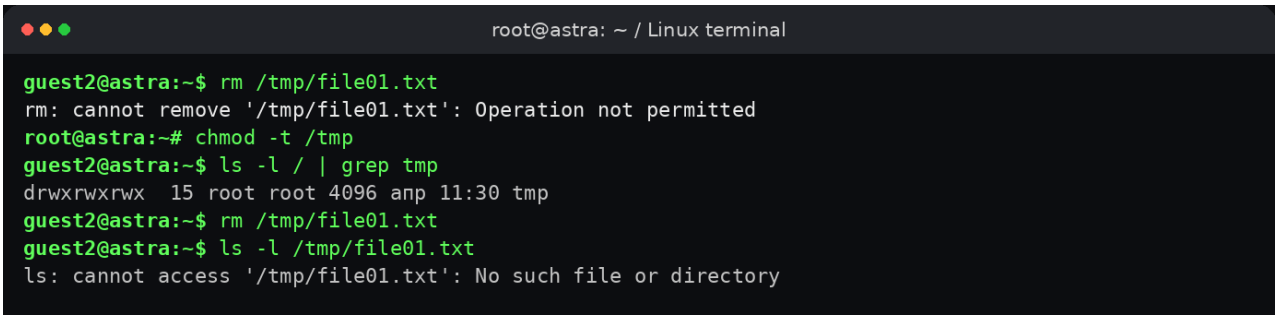
Результат выполнения команд в терминале Linux.

Наблюдение. Пользователь guest2 смог читать файл и изменять его содержимое благодаря правам доступа на сам файл.

4.8. Проверка удаления файла при наличии и отсутствии Sticky-бита

На последнем этапе было проверено, может ли пользователь, не являющийся владельцем файла, удалить его из каталога /tmp. Сначала это действие было выполнено при установленном Sticky-бите, затем Sticky-бит был снят и операция повторена.

```
rm /tmp/file01.txt
chmod -t /tmp
ls -l / | grep tmp
rm /tmp/file01.txt
```



Результат выполнения команд в терминале Linux.

Наблюдение. При установленном Sticky-бите удалить файл не удалось. После снятия бита t пользователь guest2 смог удалить чужой файл.

5. Пояснение используемых команд

• Команда `chown root:guest file` меняет владельца файла на пользователя root и группу файла на guest.

• Команда `chmod u+s file` устанавливает SetUID-бит, из-за чего исполняемый файл запускается с эффективным UID владельца файла.

• Команда `chmod g+s file` устанавливает SetGID-бит, и программа получает эффективный GID группы файла.

• Команда `chmod -t /tmp` снимает Sticky-бит с каталога. При отсутствии этого бита пользователи, имеющие права на каталог, могут удалять чужие файлы.

• Команда `chmod +t /tmp` возвращает Sticky-бит и восстанавливает защиту от удаления файлов посторонними пользователями.

6. Выводы

В ходе выполнения лабораторной работы были исследованы дополнительные атрибуты защиты в Linux: SetUID, SetGID и Sticky-бит. Установлено, что SetUID позволяет выполнять программу с эффективными правами владельца файла, а SetGID — с эффективной группой файла. На примере программы readfile было показано, что SetUID может использоваться для доступа к защищённым данным, что делает такие программы потенциально опасными при неправильной настройке. Также было установлено, что Sticky-бит защищает файлы в общедоступных каталогах, например в /tmp, от удаления пользователями, не являющимися их владельцами. Таким образом, цель работы достигнута: на практике исследовано влияние дополнительных атрибутов на разграничение доступа в ОС Linux.